Computer Programming 1

Laboratory - recursive functions (part 2)

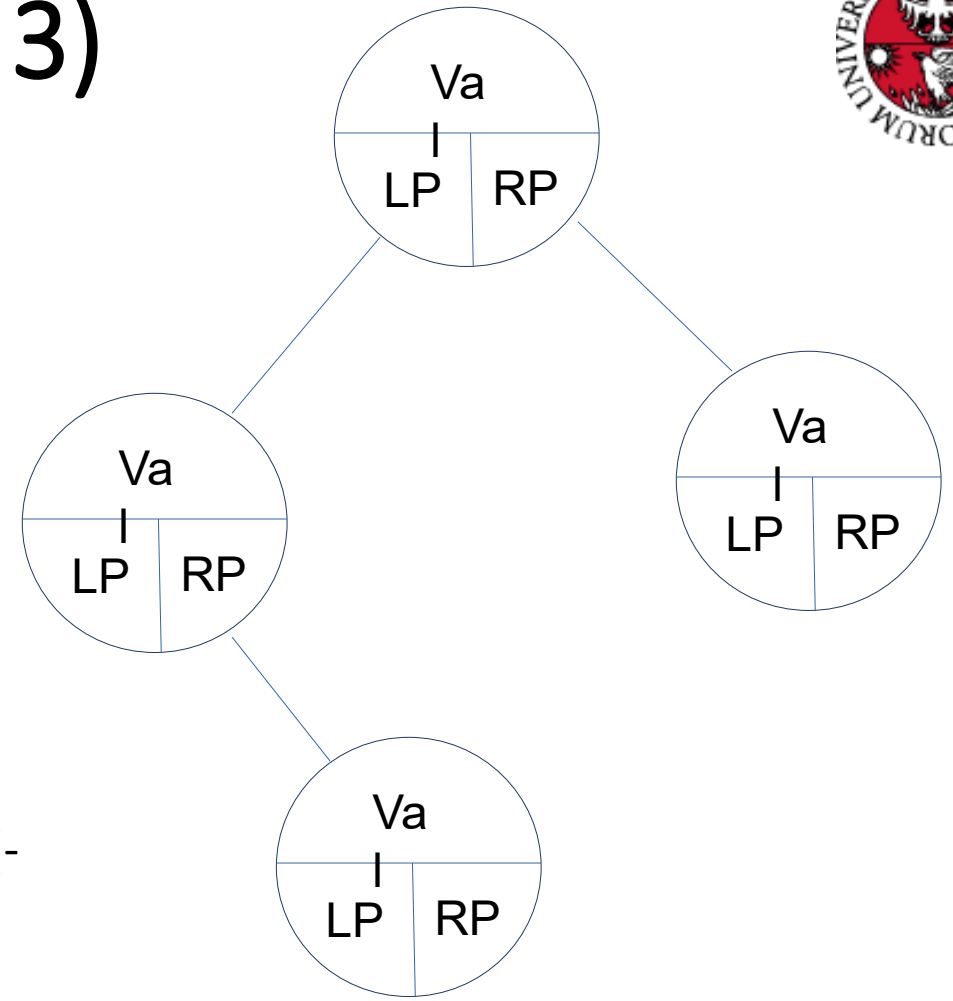
(21-10-2026)

Example 8: Binary Tree (1 out of 3)

Implement a binary tree (a tree with nodes and at maximum 2 output edges) using an array.

Hints:

- You can try with:
  - One array (it simulates memory)
  - Alternatively, with an array of arrays (one simulates pointers, the other the content of a node)
- How many elements do you need in the array storing nodes (i.e., for each node)?
  suggestion: store a node with 4 consecutive cells, such as:
    - *value*: the value of the node
    - *left pointer*: index in the array where is the left child (-1 no child)
    - *right pointer*: index in the array where is the right child (-1 no child)
    - allocation: if it is allocated (0|1)
- What functions do you need?
  suggestions:
    - *… next slide*





Example 8: Binary Tree (2 out of 3)

suggestions:

```
// initialize the array with 0
int free(int memory[])

// to add a node on the tree
int add_node(int value, int left_child, int right_child, int memory[])

// return true if the four cells in the array starting at index_of_node are marked as allocated
bool is_allocated(int index_of_node)

// getters for the node value
int get_value(int index_of_node)

// returns the index in the array where I can find the left (and right) child
int get_left(int index_of_node)
int get_right(int index_of_node)

// return true if node is a leaf
bool is_leaf(int index_of_node)
```

Example 8: Binary Tree (3 out of 3)

- Use the functions in the slide before to build a tree, for instance:
  - int memory[4 * 1000]; // space for 1000 nodes.
  - int leaf1 = add_node(10, 0, 0, memory)
  - int leaf2 = add_node(30, 0, 0, memory)
  - int leaf3 = add_node(30, leaf1, leaf2, memory)
  - int leaf4 = add_node(40, leaf3, 0, memory)

- Implement an iterative function to visit the array and print the value of the built node tree

- Implement a recursive function named in_order_visit (inspired by the one in the box) to visit the array and print the value of the built node tree

```
in_order_visit(int node) {
  if (is_leaf(node))
    return;
  else {
    in_order_visit(left_child)
    cout << "node value is:"<< get_value(node)
    in_order_visit(right_child)
  }
}
```

Example 9b: Fibonacci

- Write a program that calculates the Fibonacci numbers till a positive integer* received by the user
- Constraint: Implement a recursive function to compute the Fibonacci.
- Do not use arrays or strings, or any other complex data type.

$$F_0 = 0$$
$$F_1 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

n	*Fibonacci*
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
…	…

* condition to verify

[optional] Example 9a: Fibonacci - iterative

If you have not implemented it before, you can also implement the iterative version

Example 9c: Fibonacci – recursive v2

- Write a program that calculates the Fibonacci numbers till a positive integer* received by the user
- Constraint: Implement a **tail-recursion function**** to compute the Fibonacci.
- Do not use arrays or strings, or any other complex data type.

$$F_0 = 0$$
$$F_1 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

* condition to verify

** **You can implement the tail-recursion function version or the non-tail recursion function version, depending on the version implemented in the previous exercise (i.e., 9b)**

Note. Recursive functions can be

- *Tail-recursion*
- *Non-tail recursion*

… See the next slide

Types of recursive function (*recall*)

Recursive functions can be:

- *Tail-recursion:* the recursive call is the last operation
- *Non-tail recursion:* the recursive call is NOT the last operation, other operation are performed after the recursive call

```
// E.g., Tail-Recursion factorial
long long fact_tail(int n, long long acc) {
  if (n <= 1) {
     return acc;  // base case
  }
  // last operation, it is the recursive function
  return fact_tail(n - 1, acc * n);
}

// Wrapper function
long long fact(int n) {
   return fact_tail(n, 1);
}
```

```
// E.g., Non-Tail recursion factorial
long long fact_non_tail(int n) {
  if (n <= 1) {
     return 1;  // base case
  }
  // the recursive function is not the last operation, the
  // retuened value of the the recursive function is in fact
  // multiplied by n
  return n*fact_non_tail(n – 1);
}
```

n	*n*!
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
…	…

Example 11: power of

Write a program that implements a recursive function to calculate the power of a number given by the user*.

Constraints

- The user needs to insert two numbers: the base number and the exponent
- Do not use any other libraries than iostream
- Do not use arrays or strings, or any other complex data type

E.g.,
Input the base number: 2
Input the exponent: 3
2 raised to the power of 3 is: 8

E.g.,
Input the base number: 10
Input the exponent: 4
10 raised to the power of 4 is: 10000

* condition to verify

Example 12a: recursive MinMax

Write a program that finds the maximum and minimum numbers in an array of numbers.

- The program takes *n*, a positive integer* from the user
- The program calls a function to randomly generate an array of *n* numbers
- Then, the program calls another function to compute the maximum of the numbers in the array
- Then, the program calls another function to compute the minimum of the numbers in the array

Constraints

- Do not use any other libraries than iostream
- Use an array
- Implement two recursive functions to find the maximum and minimum of the numbers in the array

* condition to verify

Example 12b: recursive MinMax

Write a program that finds the maximum and minimum numbers in an array of numbers.

- The program takes *n*, a positive integer* from the user
- The program calls a function to randomly generate an array of *n* numbers
- Then, the program calls a function to compute the maximum and the minimum of the numbers in the array

Constraints

- Do not use any other libraries than iostream
- Use an array
- Implement a recursive function to find both the maximum and minimum of the numbers in the array, such a recursive function has to return a pair of values (i.e., the maximum and minimum)

* condition to verify